



南开大学
Nankai University

College of Software Engineering

Yoga Assistant

Final Project Report

Course: Web Application Development

Instructor: Professor BinHui Wang

Groupe Members: Bakhtiary Mohammad Fridoon
2120256010
Shah Shyam
2120256042
Muhammadi Muhammad Ali
2120256033

Date: June 16, 2026

Abstract

The Yoga Position Assistant is an innovative application designed to support yoga practitioners by providing real-time pose analysis, guidance, and feedback. The system utilizes advanced computer vision techniques, specifically MediaPipe Pose, to automatically detect body landmarks from images and videos of yoga poses. Users exercise in front of a webcam, visualize skeletal landmarks on the screen, compare their posture with reference poses, and receive feedback on alignment deviations. All pose data is stored in structured JSON files for further analysis and reference.

The application incorporates an intuitive graphical user interface (GUI) developed with PyQt5, enabling users to select poses dynamically, view real-time feedback, and save pose information efficiently. The project emphasizes the integration of a technology stack that includes Python, OpenCV, MediaPipe, and JSON data handling, ensuring a robust, cross-platform solution.

This report details the design, implementation, functionality, and deployment of the Yoga Position Assistant, highlighting its potential to enhance the practice of yoga by improving posture, alignment, and consistency. The system also provides a foundation for extensions, such as automated duration recommendations based on pose difficulty, personalized training plans.

Contents

1	Introduction	4
2	Requirements Analysis	4
2.1	Functional Requirements	4
2.2	Non-Functional Requirements	4
3	Functionality	5
3.1	Real-Time Webcam Pose Detection	5
3.2	Pose Estimation and Landmark Extraction	5
3.3	Skeleton Visualization	5
3.4	Pose Comparison and Accuracy Calculation	6
3.5	Real-Time Feedback System	6
3.6	Performance Metrics	6
3.7	Workout Program Mode	6
3.8	Reference Pose Management	7
4	Implementation Ideas	7
4.1	System Architecture	7
4.2	System Workflow	7
4.3	Pose Detection Algorithm (Pseudo-code)	8
4.4	Pose Comparison Algorithm (Pseudo-code)	8
4.5	Real-Time Processing Pipeline	8
4.6	Feedback Mechanism	9
4.7	Program Mode Logic (Workout Planner)	9
4.8	Design Rationale	9
5	Technology Stack	10
5.1	Programming Language	10
5.2	Computer Vision and Pose Detection	10
5.3	Graphical User Interface	10
5.4	Data Management	10
5.5	Mathematical and Utility Libraries	10
5.6	Development Tools	10
6	Environment Setup	10
6.1	System Requirements	11
6.2	Project Directory Structure	11
7	User Interface and Interaction	11
7.1	Main window Panels Description	11
7.2	Toolbar Buttons and Actions	12
7.3	Guided Yoga Program Interaction	12
8	Testing	12
8.1	Testing Strategy	13
8.2	Pose Detection Testing	13
8.3	Pose Matching Testing	13
8.4	Reference Pose Loading Testing	14
8.5	Graphical User Interface Testing	14
8.6	Guided Yoga Program Testing	14

8.7	Image Capture and Save Testing	14
8.8	Testing Summary	15
9	Deployment	15
9.1	Deployment via Source Code	15
10	System Documentation	15
10.1	Pose Matching Methodology	16
10.2	Reference Pose Data	16
10.3	Performance Considerations	16
11	Conclusion	16

1 Introduction

Yoga is an ancient practice that emphasizes the integration of mind, body, and breath to enhance physical health, mental well-being(6), and overall balance. Maintaining proper alignment and posture in each yoga pose is crucial to achieve the full benefits of the practice and to prevent injuries. However, for beginners and even intermediate practitioners, it can be challenging to ensure correct posture without guidance or feedback.

The Yoga Position Assistant is designed to address this challenge by providing a digital platform that assists users in learning, practicing, and refining yoga poses. The application leverages advanced computer vision techniques(5; 7), particularly the MediaPipe Pose framework(1), to detect and track body landmarks from images and videos of yoga poses. By comparing the user's posture with reference poses, the system can provide immediate visual feedback, highlighting alignment deviations and suggesting corrective actions.

This project aims to develop a user-friendly application that integrates a responsive graphical user interface (GUI) using PyQt5, enabling users to select poses, visualize skeleton landmarks, and save pose data in structured JSON format. The system is intended to support yoga practitioners of different levels, from beginners to advanced, and to facilitate self-guided practice while promoting safety and effectiveness.

The following sections of this report will detail the requirements, functionalities, implementation strategies, technology stack, and deployment considerations for the Yoga Position Assistant, providing a comprehensive overview of the design and development of the application.

2 Requirements Analysis

This section defines the functional and non-functional requirements of the Yoga Position Assistant system. The goal is to clearly identify what the system must achieve, how users will interact with it, and the constraints under which the application operates.

2.1 Functional Requirements

The proposed Yoga Pose Detection and Workout Planning System shall support both real-time and offline pose analysis. The system shall capture live webcam input and process images loaded from local directories. Using the MediaPipe Pose framework(1), it shall detect human body landmarks and visualize the corresponding skeletal structure to help users understand body alignment. Furthermore, the system shall compare detected poses with predefined reference yoga poses and evaluate posture accuracy, providing color-coded visual feedback where green indicates a correct posture, yellow indicates a nearly correct posture, and red indicates an incorrect posture.

In addition to pose analysis, the system shall provide an interactive graphical interface that allows users to select yoga poses dynamically and create customized workout plans. Users shall be able to define pose duration, rest periods, and repetition counts for each exercise. During workout sessions, the application shall display preview images of upcoming poses to assist users in preparation. The system shall also support batch processing, enabling the automatic analysis of multiple images from a selected folder, thereby enhancing usability and efficiency.

2.2 Non-Functional Requirements

The system must satisfy the following quality constraints:

- **Usability:** The application shall provide a simple and user-friendly GUI using PyQt5.

- **Performance:** The system shall process images and live webcam feed efficiently with minimal delay using optimized MediaPipe pipelines.
- **Real-Time Feedback:** The system shall provide real-time feedback on pose alignment with minimal latency.
- **Scalability:** The system architecture shall allow future expansion, such as wearable sensor integration, and cloud synchronization.
- **Reliability:** The pose detection and feedback shall be consistent and accurate across different yoga poses and input images.
- **Accessibility / Guidance:** The GUI shall clearly indicate the current pose, next pose, and rest times to effectively guide users through the workout.

3 Functionality

The Yoga Position Assistant system provides a real-time interactive environment for yoga pose detection, evaluation, and guided training. The system integrates computer vision, pose estimation, and a graphical user interface to deliver immediate feedback to users.

3.1 Real-Time Webcam Pose Detection

The system captures live video frames using an OpenCV-based webcam module. Each frame is processed in real-time and passed to the MediaPipe Pose model to extract 33 human body landmarks(1). These landmarks represent key body joints such as shoulders, elbows, hips, knees, and ankles.

The processing pipeline is implemented in the `CameraWidget` class, which continuously updates frames using a timer-based loop.

3.2 Pose Estimation and Landmark Extraction

MediaPipe Pose is used as the core pose estimation engine. For each incoming frame:

- The image is converted from BGR to RGB format.
- Pose landmarks are extracted using the MediaPipe model.
- Each landmark contains normalized (x, y) coordinates and visibility confidence.

The extracted landmarks are used for both visualization and pose comparison.

3.3 Skeleton Visualization

The system overlays two types of skeletons on the video feed:

- **User Skeleton:** Represents the real-time detected pose from webcam input.
- **Reference Skeleton:** Loaded from predefined JSON pose files and rendered in gray color.

Connections between joints are drawn using OpenCV lines, while joints are represented using circular markers.

3.4 Pose Comparison and Accuracy Calculation

The system compares user pose landmarks with reference pose landmarks using a distance-based similarity metric.

- Euclidean distance is calculated between corresponding joints(8).
- Each landmark is assigned a score based on threshold values:
 - Green: Correct alignment (low distance)
 - Yellow: Near correct alignment
 - Red: Incorrect alignment (high distance)
- Overall pose accuracy is computed as the average normalized score across all matched landmarks.

This logic is implemented in the `compare_pose_landmarks()` function inside `pose_utils.py`.

3.5 Real-Time Feedback System

The system provides immediate visual feedback to the user:

- Correct body parts are highlighted in green.
- Slight deviations are highlighted in yellow.
- Incorrect posture regions are highlighted in red.

This allows users to adjust their posture dynamically during practice.

3.6 Performance Metrics

The system continuously computes and displays:

- Frames per second (FPS) for performance monitoring.
- MediaPipe confidence score based on landmark visibility.
- Pose matching accuracy based on similarity with reference pose.

These metrics are emitted from the `CameraWidget` using Qt signals and displayed in the GUI status panel.

3.7 Workout Program Mode

The system includes a structured workout planner that allows users to:

- Select difficulty levels (Beginner, Intermediate, Advanced).
- Define hold time for each pose.
- Define rest time between poses.
- Set number of repetitions.

A built-in timer manages pose execution phases and provides real-time countdown feedback.

3.8 Reference Pose Management

Reference yoga poses are stored in JSON format and loaded dynamically using the **PoseManager** module. Each pose contains normalized landmark coordinates that are scaled according to the video frame size during rendering.

4 Implementation Ideas

The Yoga Position Assistant system is implemented using a modular architecture that separates responsibilities into distinct components such as pose detection, rendering, pose comparison, and user interface management. The system is designed for real-time processing of webcam input and provides continuous feedback to the user.

4.1 System Architecture

The system consists of the following main modules:

- **Camera Widget (CameraWidget):** Handles webcam input, frame processing, FPS calculation, and GUI rendering.
- **Pose Detector (PoseDetector):** Uses MediaPipe to extract human body landmarks from each frame.
- **Pose Renderer (PoseRenderer):** Draws both user and reference skeletons and applies color-coded feedback.
- **Pose Utilities (pose_utils):** Provides mathematical functions for comparison, distance calculation, and scoring.
- **Pose Manager (PoseManager):** Loads reference poses and instructions from JSON files.

This modular structure ensures separation of concerns and allows future extensibility.

4.2 System Workflow

The overall workflow of the system is as follows:

1. The webcam captures a live video frame.
2. The frame is passed to the MediaPipe Pose model for landmark detection.
3. Detected landmarks are extracted and normalized.
4. The reference pose is loaded from JSON format.
5. User pose and reference pose are compared using Euclidean distance.
6. Each landmark is assigned a color based on similarity:
 - Green: correct alignment
 - Yellow: near correct
 - Red: incorrect alignment
7. The skeletons are rendered on the frame.
8. Performance metrics (FPS, confidence, accuracy) are updated in real time.
9. The processed frame is displayed in the GUI.

4.3 Pose Detection Algorithm (Pseudo-code)

```
function PROCESS_FRAME(frame):
    frame_rgb = convert_BGR_to_RGB(frame)
    results = MediaPipePose.process(frame_rgb)

    if results contains landmarks:
        user_landmarks = extract_landmarks(results)

        confidence = average_visibility(user_landmarks)

        reference_pose = load_reference_pose()

        colors, accuracy = compare_pose_landmarks(
            reference_pose,
            user_landmarks
        )

        frame = draw_reference_skeleton(frame, reference_pose)
        frame = draw_user_skeleton(frame, user_landmarks, colors)

    fps = compute_fps()
    return frame, accuracy, confidence, fps
```

4.4 Pose Comparison Algorithm (Pseudo-code)

```
function COMPARE_POSE_LANDMARKS(reference, user):
    scores = []
    colors = {}

    for each landmark in reference:
        if landmark exists in user:
            distance = Euclidean(user[landmark], reference[landmark])

            if distance < GREEN_THRESHOLD:
                color = GREEN
                score = 1.0
            else if distance < YELLOW_THRESHOLD:
                color = YELLOW
                score = 0.5
            else:
                color = RED
                score = 0.0

            colors[landmark] = color
            scores.append(score)

    accuracy = average(scores) * 100
    return colors, accuracy
```

4.5 Real-Time Processing Pipeline

The system operates in a continuous loop inside the `CameraWidget` class:

```

while camera_is_running:
    frame = capture_webcam_frame()

    results = pose_detector.process(frame)

    confidence = compute_visibility(results)

    frame = pose_renderer.draw_reference_pose(frame)

    frame, accuracy = pose_renderer.draw_user_pose(
        frame,
        results,
        mp_pose
    )

    update_fps()

    emit_GUI_signals(fps, confidence, accuracy)

    display_frame(frame)

```

4.6 Feedback Mechanism

The feedback system is based on distance thresholds:

- If landmark error ≤ 0.06 : Green (Correct posture)
- If $0.06 < error \leq 0.12$: Yellow (Minor correction needed)
- If error > 0.12 : Red (Incorrect posture)

This mechanism provides intuitive visual guidance for users during yoga practice.

4.7 Program Mode Logic (Workout Planner)

The workout planner follows a structured state machine:

```

states = [HOLD, REST, NEXT_POSE]

for each repetition:
    for each pose:
        enter HOLD state
        countdown hold_time

        enter REST state
        show next_pose_preview
        countdown rest_time

```

4.8 Design Rationale

The system is designed with modularity and real-time performance in mind. MediaPipe ensures fast and accurate pose estimation, while PyQt5 provides an interactive and responsive user interface. The separation between detection, rendering, and UI layers allows easy maintenance and future enhancements such as AI-based correction suggestions or wearable sensor integration.

5 Technology Stack

The Yoga Position Assistant system relies on a combination of Python libraries, frameworks, and tools to achieve real-time pose detection, rendering, and user interaction. The technology stack is chosen to balance performance, modularity, and cross-platform compatibility.

5.1 Programming Language

- **Python 3.10+** – The primary language for implementing all modules. Python is chosen for its extensive library ecosystem, ease of use, and rapid prototyping capabilities.

5.2 Computer Vision and Pose Detection

- **OpenCV (cv2)** – Handles webcam video capture, frame preprocessing, image conversion, and rendering of graphical elements(2).
- **MediaPipe** – Provides state-of-the-art human pose estimation with pre-trained models, including landmark detection and smoothing for real-time applications.

5.3 Graphical User Interface

- **PyQt5** – A Python binding for the Qt framework, used to develop the interactive GUI including camera display, status panel, controls for program selection, and real-time feedback(3).

5.4 Data Management

- **JSON** – Stores reference pose landmarks and instructional metadata. Enables easy addition of new yoga poses without modifying source code(4).
- **OS and Path Libraries** – For managing file paths, directories, and cross-platform compatibility in accessing assets.

5.5 Mathematical and Utility Libraries

- **math** – Provides mathematical operations for computing Euclidean distances, angles, and threshold-based scoring.

5.6 Development Tools

- **IDE: VSCode** – For code editing, debugging, and project management.
- **Version Control: Git** – Ensures proper source code tracking and collaborative development.
- **Python Package Management: pip / virtualenv** – For managing dependencies and isolating project environments.

6 Environment Setup

This section describes the required environment and step-by-step setup process for running the Yoga Pose Assistant application. The system is implemented in Python and depends on several computer vision and GUI libraries.

6.1 System Requirements

The application requires the following minimum hardware and software configuration:

- **Operating System:** Windows 10/11
- **Python Version:** 3.9, 3.10, 3.11, or 3.12
- **RAM:** Minimum 8 GB recommended
- **Camera:** Built-in or external webcam (HD recommended)
- **Processor:** Dual-core CPU or higher

6.2 Project Directory Structure

The system follows a modular directory structure:

```
yoga_pose_assistant/  
  main.py  
  requirements.txt  
  README.md  
  core/  
    pose_detector.py  
    pose_manager.py  
    pose_renderer.py  
    pose_utils.py  
  ui/  
    main_window.py  
    camera_widget.py  
    status_panel.py  
    pose_selection_panel.py  
    instruction_panel.py  
    user_guide_dialog.py  
  assets/  
    reference_poses/  
  data/  
    captured_images/  
    poses/
```

7 User Interface and Interaction

The Yoga Pose Assistant provides an intuitive and user-friendly graphical interface designed with **PyQt5**. The interface is divided into three main sections, each serving a distinct function in guiding the user through yoga practice.

7.1 Main window Panels Description

- **Pose Selection Panel (Left):** Users can browse through 28 available yoga poses categorized by difficulty: Beginner, Intermediate, Advanced. Selecting a pose loads the corresponding reference skeleton and instructions.
- **Camera View Panel (Center):** Displays the live webcam feed along with overlaid skeletons:

- Gray skeleton – target pose reference
- Colored skeleton – user pose, landmarks color-coded by alignment:
 - * Green: landmark closely matches reference
 - * Yellow: small adjustment needed
 - * Red: far from reference

The bottom section displays pose instructions and step-by-step guidance.

- **Status Panel (Right):** Provides real-time feedback including:
 - **Current Pose**
 - **Reference Image** of the pose
 - **FPS** of the camera
 - **Pose Matching Score** (0–100%)
 - **MediaPipe Accuracy Score** (0–100%)
 - **Program Timer** for guided sessions

7.2 Toolbar Buttons and Actions

Button	Action
Start Camera	Opens the webcam and starts pose detection in real-time.
Stop Camera	Stops webcam feed and releases the camera.
Capture	Freezes the current frame with overlayed skeletons.
Save	Saves the last captured frame to.
User Guide	Opens an in-app help dialog for instructions and tips.

7.3 Guided Yoga Program Interaction

Users can start a guided yoga session from the Status Panel:

1. Select a **Program Level**: Beginner, Intermediate, or Advanced.
2. Configure **Hold Time** (seconds per pose), **Rest Time** (seconds between poses), and **Repetitions**.
3. Click **Start Program** to begin an automated session:
 - Poses are cycled automatically with countdown timers.
 - Skeleton overlay pauses during rest periods.
 - Preview of the next pose may appear.
4. Click **Stop Program** to end the session at any time.

8 Testing

Testing was conducted to evaluate the correctness, reliability, and usability of the Yoga Pose Assistant application. The objective was to verify that the system accurately detects body landmarks, compares user poses with reference poses, provides meaningful visual feedback, and operates smoothly during guided yoga sessions.

8.1 Testing Strategy

The application was tested using a combination of functional testing, integration testing, and user interface testing. Each major module was evaluated independently before testing the complete system workflow.

The following components were tested:

- Pose Detection Module
- Pose Matching Module
- Reference Pose Loading Module
- Graphical User Interface
- Guided Yoga Program Module
- Image Capture and Save Functions

8.2 Pose Detection Testing

The MediaPipe Pose framework was tested using live webcam input under different environmental conditions.

Test Case	Expected Result	Result
Full body visible in camera frame	Complete landmark detection	Passed
Partial body visible	Reduced landmark detection accuracy	Passed
Good lighting conditions	Stable tracking	Passed
Poor lighting conditions	Lower confidence values	Passed
Rapid body movement	Temporary tracking fluctuations	Passed

Table 1: Pose Detection Testing

The tests showed that MediaPipe successfully detected body landmarks in real time when the user's full body was visible. Detection confidence decreased under poor lighting conditions and during rapid movements, which is consistent with the limitations of computer vision systems.

8.3 Pose Matching Testing

The pose comparison algorithm was tested using multiple reference poses and user poses.

The system computes the Euclidean distance between corresponding landmarks and assigns color-coded feedback according to predefined thresholds.

Distance Range	Color	Interpretation
Distance ≤ 0.06	Green	Correct alignment
$0.06 < \text{Distance} \leq 0.12$	Yellow	Minor adjustment needed
Distance > 0.12	Red	Significant correction required

Table 2: Landmark Evaluation Criteria

The accuracy score was calculated as the average landmark score multiplied by 100. Testing confirmed that accuracy values increased when users aligned their body closer to the reference pose.

8.4 Reference Pose Loading Testing

Reference pose files stored in JSON format were tested to ensure correct loading and visualization. The system successfully loaded pose landmarks and instructional information from the JSON database and displayed them in the interface.

8.5 Graphical User Interface Testing

The user interface components were tested individually and as an integrated system.

Interface Component	Result
Pose selection panel	Passed
Camera start/stop controls	Passed
Live accuracy updates	Passed
Reference image preview	Passed
Instruction display panel	Passed
Status panel updates	Passed

Table 3: GUI Testing Results

The graphical interface remained responsive during real-time pose detection and correctly updated all metrics and visual components.

8.6 Guided Yoga Program Testing

The guided yoga program functionality was tested using different program configurations.

Feature	Result
Program level selection	Passed
Hold time countdown	Passed
Rest time countdown	Passed
Automatic pose transitions	Passed
Repetition handling	Passed
Program termination	Passed

Table 4: Guided Program Testing

Testing confirmed that the timer system correctly managed pose durations, rest periods, and repetitions while updating the user interface accordingly.

8.7 Image Capture and Save Testing

The capture and save functionality was evaluated to ensure that users could preserve their practice sessions.

Operation	Result
Capture current frame	Passed
Freeze display after capture	Passed
Save image to local storage	Passed
Save multiple captures	Passed

Table 5: Capture and Save Testing

The system successfully stored captured images in the designated output directory while preserving the visual overlays.

8.8 Testing Summary

Overall testing demonstrated that the Yoga Pose Assistant performs reliably in real-time environments. The pose detection module accurately identifies body landmarks, the pose matching algorithm provides meaningful feedback through color-coded visualization and accuracy scores, and the user interface supports smooth interaction throughout yoga practice sessions. The guided yoga program and image capture functionalities also operated as expected, confirming the readiness of the application for practical use.

9 Deployment

The Yoga Pose Assistant is a Python desktop application. Deployment involves preparing the environment, packaging dependencies, and distributing the application for end users. The following steps outline deployment options for Windows, macOS, and Linux.

9.1 Deployment via Source Code

End users can run the application directly from source:

1. Clone or download the repository:

```
git clone <repository-url>
cd yoga_pose_assistant
```

2. Create and activate a virtual environment:

```
python -m venv venv
# Windows:
.\venv\Scripts\activate
# macOS / Linux:
source venv/bin/activate
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Run the application:

```
python main.py
```

10 System Documentation

The Yoga Pose Assistant is a real-time computer vision-based desktop application designed to support yoga practitioners through automated pose detection, evaluation, and guided training sessions. The system integrates MediaPipe pose estimation with a PyQt5-based graphical interface to provide immediate feedback on user posture.

10.1 Pose Matching Methodology

Pose similarity is computed using Euclidean distance between corresponding landmarks(8):

$$d = \sqrt{(x_u - x_r)^2 + (y_u - y_r)^2}$$

where x_u, y_u represent user landmarks and x_r, y_r represent reference landmarks. Each landmark is classified based on threshold values:

- $d \leq 0.06$: Correct alignment (Green)
- $0.06 < d \leq 0.12$: Moderate deviation (Yellow)
- $d > 0.12$: Incorrect alignment (Red)

The final accuracy score is computed as:

$$Accuracy = \frac{1}{N} \sum_{i=1}^N score_i \times 100$$

10.2 Reference Pose Data

Reference poses are stored in JSON format and contain normalized landmark coordinates in the range [0, 1]. The system supports two formats:

```
{ "landmarks": { "nose": [0.15, 0.31], ... } }
```

or

```
{ "nose": [0.15, 0.31], ... }
```

10.3 Performance Considerations

To ensure real-time performance, the system:

- Uses lightweight MediaPipe models optimized for CPU execution.
- Processes frames in a continuous event-driven loop.
- Computes FPS, confidence, and accuracy metrics in real time.
- Utilizes PyQt signals for efficient GUI updates.

11 Conclusion

This project presented the design and implementation of the **Yoga Pose Assistant**, a desktop application developed to support yoga practitioners through real-time pose analysis and interactive feedback. The primary objective of the system was to help users perform yoga poses more accurately by comparing their body posture against predefined reference poses and providing immediate visual guidance.

The application successfully integrates several modern technologies, including **MediaPipe Pose** for human landmark detection, **OpenCV** for real-time image processing, and **PyQt5** for creating an intuitive graphical user interface. Through these technologies, the system is capable of detecting body landmarks from a live webcam feed, visualizing both reference and user skeletons, calculating pose-matching accuracy, and providing color-coded feedback that enables users to identify and correct posture deviations.

One of the key contributions of the project is the implementation of a pose comparison mechanism based on landmark distance analysis. By evaluating the similarity between user landmarks and reference landmarks, the application provides an easily understandable accuracy score and visual indicators that assist users in improving their alignment. In addition, the guided yoga program feature extends the functionality of the system by allowing users to create structured training sessions with customizable hold times, rest intervals, and repetitions.

Testing results demonstrated that the system performs reliably under normal operating conditions and is capable of delivering responsive real-time feedback. The modular architecture adopted during development also improves maintainability and allows new poses, instructions, and features to be integrated with minimal modifications to the existing codebase.

Although the current implementation achieves its primary objectives, several opportunities for future enhancement exist. Future versions may incorporate machine learning models for more advanced pose evaluation, personalized training recommendations, automatic correction suggestions for specific body parts, progress tracking and performance analytics, cloud-based data synchronization, and integration with wearable sensors for more comprehensive movement analysis.

In conclusion, the Yoga Pose Assistant demonstrates how computer vision technologies can be effectively applied to fitness and wellness applications. The system provides an accessible, interactive, and practical tool for yoga practitioners while establishing a strong foundation for future research and development in intelligent posture assessment systems.

References

- [1] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C., Yong, M., Lee, J., et al. *MediaPipe: A Framework for Building Perception Pipelines*. Google Research, 2019. Available at: <https://mediapipe.dev>
- [2] Bradski, G. *The OpenCV Library*. Dr. Dobb’s Journal of Software Tools, 2000. Available at: <https://opencv.org>
- [3] Summerfield, M. *Rapid GUI Programming with Python and Qt*. Prentice Hall, 2007.
- [4] Bray, T. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259, Internet Engineering Task Force (IETF), 2017.
- [5] Szeliski, R. *Computer Vision: Algorithms and Applications*. Springer, 2nd Edition, 2022.
- [6] Iyengar, B. K. S. *Light on Yoga*. HarperCollins Publishers, 2006.
- [7] Cao, Z., Hidalgo, G., Simon, T., Wei, S., and Sheikh, Y. *OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019.
- [8] Deza, E., and Deza, M. *Encyclopedia of Distances*. Springer, 4th Edition, 2016.